

Arduino

I2C Protocol & Multiple Devices

Lesson Handout

In this lesson you will learn how Arduino uses the I2C protocol to communicate with multiple devices using only TWO wires — SDA and SCL. You will discover device addresses, master/slave roles, and build your own I2C circuit!

■ Learning Objectives

- Understand what I2C is and why it needs only 2 wires
- Identify SDA (data) and SCL (clock) pins on Arduino UNO
- Explain how 7-bit addresses let multiple devices share one bus
- Describe the Master → Slave communication flow
- Connect an I2C sensor and display to Arduino in a circuit

■ 1 — What is I2C?

I2C stands for **Inter-Integrated Circuit**. It is a serial communication protocol invented by **Philips in 1982** that allows a single Master device to communicate with many Slave devices — all sharing just **two wires**. Think of it like a classroom: the teacher (Master) calls a student's name (address), and only that student (Slave) responds.

Wire	Pin (UNO)	Purpose
SDA	A4	Serial Data Line — carries the actual bits
SCL	A5	Serial Clock Line — synchronises timing

■ 2 — Why Use I2C?

■ Saves wires	Only SDA + SCL needed — no separate wires per device
■ Easy to expand	Up to 127 devices can share the same two wires
■ Built into Arduino	Wire library included — no extra hardware needed
■ Addresses	Each device has a unique 7-bit address (e.g. 0x27, 0x48)

■ 3 — Key Terms

Master	The controller device that starts communication and generates the clock. Usually the Arduino.
Slave	A peripheral device that listens and responds only when its address is called.
SDA	Serial Data Line — carries data bits between Master and Slave.
SCL	Serial Clock Line — Master controls the speed; all devices sync to it.
Address	A 7-bit number unique to each Slave (e.g. 0x27 for LCD, 0x48 for sensor).
ACK	Acknowledgement — a 1-bit reply confirming each byte was received correctly.
NACK	No-Acknowledgement — signals an error or end of data transfer.
START/STOP	Special signal conditions that mark the beginning and end of a transaction.
Wire.h	Arduino's built-in I2C library. Use Wire.begin(), Wire.requestFrom(), etc.

■ 4 — I2C Communication Flow

Every I2C **read transaction** follows this exact sequence of 6 steps:

Step 1	START	Master pulls SDA LOW while SCL stays HIGH — signals bus is busy
Step 2	ADDRESS + R/W bit	Master sends 7-bit slave address + 1 bit (1=Read, 0=Write)
Step 3	Slave ACK	Matching slave pulls SDA LOW to acknowledge its address
Step 4	Data Transfer	Slave sends data bytes one at a time, MSB first

Step 5	Master ACK/NACK	Master sends ACK after each byte; NACK after the last byte
Step 6	STOP	Master releases SDA HIGH while SCL stays HIGH — bus is free

■ 5 — Arduino Code Snippets

Reading from an I2C sensor (Wire library):

```
#include <Wire.h>

void setup() {
  Wire.begin();          // Join I2C bus as Master
  Serial.begin(9600);
}

void loop() {
  Wire.requestFrom(0x48, 2); // Request 2 bytes from device 0x48
  while (Wire.available()) {
    int data = Wire.read(); // Read each byte
    Serial.println(data);
  }
  delay(500);
}
```

Writing to an I2C LCD display:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2); // Address 0x27, 16 cols, 2 rows

void setup() {
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Hello I2C!");
}
```

■■ 6 — Common I2C Device Addresses

Device	Default Address	Notes
LCD 16×2 (PCF8574)	0x27 or 0x3F	Check solder jumpers on back
OLED 128×64 (SSD1306)	0x3C or 0x3D	Depends on SA0 pin

Temp Sensor (LM75)	0x48 – 0x4F	3 address pins configurable
RTC DS3231	0x68	Fixed — cannot be changed
Accelerometer MPU-6050	0x68 or 0x69	AD0 pin sets address
EEPROM 24C32	0x50 – 0x57	A0–A2 pins configurable

■ 7 — Drag-and-Drop Activities (Digital Companion)

These three activities match the interactive HTML file. Use them as reference or as a paper version of the exercises.

1 Pin Role Matcher — Draw lines to match each term to its description

- SDA ☐ Serial clock line synchronising all devices
- SCL ☐ Serial data line carrying bits between devices
- Master ☐ Listens and responds only when addressed
- Slave ☐ Initiates communication and generates the clock
- Address ☐ 1-bit acknowledgement after each byte received
- ACK ☐ 7-bit unique identifier for each device on the bus

2 Circuit Builder — Write the correct component in each slot

- Power rail (5V) ☐ [_____]
- SDA pull-up resistor to VCC ☐ [_____]
- SCL pull-up resistor to VCC ☐ [_____]
- Peripheral #1 address 0x27 ☐ [_____]
- Peripheral #2 address 0x48 ☐ [_____]
- Common ground rail ☐ [_____]

3 Sequence Chain — Number these I2C steps 1–6 in correct order

- ☐ Master sends STOP condition
- ☐ Slave transmits data bytes
- ☐ Master sends START condition
- ☐ Master sends ACK / NACK
- ☐ Master sends 7-bit address + READ bit
- ☐ Slave sends ACK (address match)

8 — Quick Quiz

Q1. How many wires does I2C use to communicate?

- | | | | |
|------|--------|------|------|
| a) 1 | b) 2 ✓ | c) 4 | d) 8 |
|------|--------|------|------|

Q2. Which Arduino UNO pin is SDA?

- | | | | |
|-------|--------|---------|-------|
| a) D2 | b) D13 | c) A4 ✓ | d) A0 |
|-------|--------|---------|-------|

Q3. What is the role of the Master?

- | | | | |
|----------------------------|------------------------------|----------------|---------|
| a) Listens for its address | b) Initiates communication ✓ | c) Stores data | d) None |
|----------------------------|------------------------------|----------------|---------|

Q4. What does ACK mean?

- | | | | |
|----------------------|----------------------|-----------------|-------------------|
| a) Address Clock Key | b) Acknowledgement ✓ | c) Analog Clock | d) Access Control |
|----------------------|----------------------|-----------------|-------------------|

Q5. What is the typical address of an I2C LCD?

- | | | | |
|---------|---------|-----------|---------|
| a) 0x00 | b) 0xFF | c) 0x27 ✓ | d) 0x10 |
|---------|---------|-----------|---------|

9 — Tips for Working with I2C

- Always connect SDA→A4 and SCL→A5 on Arduino UNO (or SDA/SCL pins on Mega/Nano).
- Add 4.7 kΩ pull-up resistors from SDA and SCL to 5 V — required for reliable comms.
- Use the I2C scanner sketch to find unknown device addresses on your bus.
- Include the Wire.h library before any I2C calls — it is built into Arduino IDE.
- If two devices share an address, check if the device has address-config pins (A0–A2).

- Keep I2C wires short (< 1 m) to avoid signal degradation at higher speeds.
- Fun fact: I2C was invented by Philips in 1982; over 100 types of devices use it today!

■ 10 — Further Exploration

Try these project ideas to practise your I2C skills:

- ■ Connect a BMP280 temperature & pressure sensor — display readings on OLED
- Wire up a DS3231 RTC clock module — build a digital clock on LCD
- Use MPU-6050 accelerometer — detect tilt/shake gestures
- ■ Chain three I2C displays at different addresses — show different data on each
- Write an I2C scanner: scan addresses 0x03–0x77 and print which respond

Keep Exploring! ■

I2C is just the beginning — explore SPI, UART, and Bluetooth next!